



Universitatea *Transilvania* din Braşov
Facultatea de Matematică şi Informatică
Departamentul de Matematică şi Informatică

Implementarea puzzle-ului Sudoku în Common Lisp şi Haskell Documentaţie pentru proiect Programare Funcţională

Autori:

Hurubă Adriana
Urs Ruxandra-Mihaela
Vaida Raluca
Vîlcu Andreea

aprilie 2026

Cuprins

1	Introducere	2
1.1	Despre Sudoku	2
2	Limbajul Common Lisp	2
3	Limbajul Haskell	2
4	Logica de implementare	3
4.1	Implementarea în Common Lisp	3
4.1.1	Regulile de bază și validarea	3
4.1.2	Identificarea celulelor libere	3
4.1.3	Algoritmul de Backtracking și funcția <code>solve</code>	3
4.1.4	Motivația utilizării Backtracking-ului	4
4.1.5	Metoda de generare a unui puzzle: <code>generator.lisp</code>	4
4.1.6	Randomizarea și algoritmul de amestecare	4
4.1.7	Strategia de generare a soluției complete	4
4.1.8	Transformarea soluției în puzzle	5
4.1.9	Puzzle demo	5
4.2	Implementarea în Haskell	5
4.2.1	Reprezentarea datelor și validarea regulilor	5
4.2.2	Backtracking recursiv: <code>Maybe Monad</code> și <code>Pattern Matching</code>	6
4.2.3	Generarea puzzle-urilor: randomizare și <code>IO Monad</code>	6
4.2.4	Claritatea codului și apropierea de specificația matematică	7
4.2.5	Alte observații și detalii de implementare	7
5	Comparații între abordarea Common Lisp și Haskell	8
5.1	Modul de rezolvare prin Backtracking	8
5.2	Comparație: tratarea succesului/eșecului	8
6	Testarea sistemului	9
6.1	Suita de teste	9
6.1.1	Teste Common Lisp	9
6.1.2	Teste Haskell	10
6.2	Rezultate și execuție	11
6.2.1	Rezultate Common Lisp (SBCL 2.6.3)	11
6.2.2	Rezultate Haskell (GHC 9.6.7)	11
6.3	Evaluare comparativă și concluzii	12

1 Introducere

Proiectul are ca obiectiv implementarea unui sistem complet pentru generarea și rezolvarea puzzle-urilor **Sudoku** (de dimensiuni 4x4 și 9x9). Dincolo de latura pur algoritmică a rezolvării unui joc logic, scopul principal al acestei lucrări este unul de analiză comparativă. Proiectul explorează modul în care aceeași problemă computațională este modelată și rezolvată folosind două limbaje de programare distincte din sfera funcțională: Common Lisp și Haskell.

1.1 Despre Sudoku

Sudoku este un puzzle de logică ce presupune completarea unei grile cu cifre (clasic de dimensiuni 9x9, dar și variante reduse precum 4x4). Regula fundamentală a jocului este simplă: fiecare rând, fiecare coloană și fiecare sub-grilă delimitată (exemplu: 2x2 sau 3x3) trebuie să conțină toate cifrele specifice dimensiunii (de la 1 la 4, respectiv de la 1 la 9) exact o singură dată. Orice puzzle valid pornește cu un set de celule deja completate, iar jucătorul trebuie să deducă prin eliminare și logică poziția corectă pentru restul cifrelor goale prin regula precizată.

2 Limbajul Common Lisp

Common Lisp este un limbaj de programare puternic și versatil, caracterizat de o sintaxă bazată pe expresii simbolice (S-expresii), în care codul este structurat sub formă de liste. O proprietate definitorie a acestui limbaj este tratarea unitară a codului și a datelor, facilitate care permite generarea și manipularea programatică a propriului cod. Deși permite abordări variate (inclusiv procedurale sau orientate pe obiect), în cadrul acestui proiect limbajul a fost utilizat strict pentru aplicarea principiilor programării funcționale. În plus, mediul său interactiv (REPL) permite testarea și modificarea codului în timp real, oferind un avantaj semnificativ în etapa de dezvoltare și depanare.

3 Limbajul Haskell

Haskell este un limbaj de programare pur funcțional. O trăsătură fundamentală a acestui limbaj este separarea strictă a logicii pure de efectele secundare, acestea din urmă fiind izolate și gestionate prin intermediul abstractizărilor matematice denumite Monade. Această arhitectură încurajează un stil de programare declarativ, în care dezvoltatorul descrie definiția problemei și „ce” trebuie calculat, delegând mecanismele de iterare compilatorului. Datorită siguranței oferite de sistemul său de tipuri și a instrumentelor precum *Pattern Matching*, Haskell permite scrierea unui cod extrem de concis și robust, ideal pentru modelarea algoritmică a spațiilor de soluții.

4 Logica de implementare

4.1 Implementarea în Common Lisp

Nucleul rezolvării problemei date în este reprezentat de fișierul `sudoku.lisp`, care definește structura logică și mecanismul de rezolvare a puzzle-ului. Implementarea respectă principiile programării funcționale, utilizând structuri de date imutabile și recursivitate pentru explorarea soluțiilor.

4.1.1 Regulile de bază și validarea

Validarea tablei de joc se bazează pe trei reguli fundamentale ale Sudoku-ului, implementate prin funcții dedicate care verifică unicitatea unei valori:

- `valid-row?`: Verifică dacă o valoare există deja pe rândul specificat.
- `valid-col?`: Extrage coloana respectivă sub formă de listă și verifică prezența duplicatelor.
- `valid-box?`: Calculează coordonatele sub-grilei corespunzătoare (2x2 pentru 4x4 sau 3x3 pentru 9x9) și verifică dacă valoarea respectă constrângerea de grup.

Funcția principală `valid?` combină aceste verificări pentru a determina dacă plasarea unei cifre într-o anumită celulă este permisă conform regulamentului.

4.1.2 Identificarea celulelor libere

Pentru a avansa în rezolvare, sistemul trebuie să identifice pozițiile care necesită completare. Funcția `find-empty` parcurge tabla de joc (reprezentată ca o listă de liste) și returnează prima celulă care conține valoarea 0. Aceasta returnează o pereche de tip *cons cell* (`row . col`), oferind coordonatele precise ale spațiului gol, sau `nil` în cazul în care tabla este completată integral.

4.1.3 Algoritmul de Backtracking și funcția solve

Rezolvarea propriu-zisă este realizată de funcția `solve`, care aplică tehnica *backtracking* într-un mod recursiv. Procesul urmează pașii următori:

1. Se caută o celulă goală folosind `find-empty`. Dacă nu se găsește niciuna, tabla este considerată rezolvată și este returnată ca soluție finală.
2. Dacă există o celulă liberă, se iterează prin valorile posibile (de la 1 la dimensiunea tablei).
3. Pentru fiecare valoare, se verifică validitatea acesteia prin funcția `valid?`. Dacă este validă, se generează o nouă tablă prin funcția `set-value` și se apelează recursiv funcția `solve`.
4. Dacă un apel recursiv returnează o soluție, aceasta este propagată spre nivelurile superioare. În caz contrar (eșec), algoritmul „se întoarce” și încearcă următoarea valoare disponibilă.

4.1.4 Motivația utilizării Backtracking-ului

Alegerea algoritmului de *backtracking* este justificată de natura problemei Sudoku, care este o problemă de satisfacere a constrângerilor într-un spațiu de căutare vast. Motivul principal al utilizării acestei metode este eficiența în explorarea sistematică a posibilităților:

- **Explorare sistematică:** permite parcurgerea tuturor configurațiilor potențiale fără a genera combinații evident invalide.
- **Pruning:** algoritmul abandonează imediat o ramură de căutare de îndată ce o regulă este încălcată, reducând drastic numărul de stări analizate față de o căutare de tip *brute force*.
- **Garanția soluției:** backtracking-ul garantează găsirea unei soluții dacă aceasta există, fiind ideal pentru puzzle-uri cu constrângeri stricte.

4.1.5 Metoda de generare a unui puzzle: `generator.lisp`

Fișierul `generator.lisp` extinde capabilitățile sistemului prin implementarea logicii necesare pentru crearea automată de puzzle-uri Sudoku de dimensiuni variabile: de 4 linii x 4 coloane și de 9 linii x 9 coloane. Acesta utilizează funcțiile de bază definite în `sudoku.lisp` pentru a asigura validitatea și rezolvabilitatea fiecărei table generate.

4.1.6 Randomizarea și algoritmul de amestecare

Elementul central al generatorului este funcția `shuffle-list`, care implementează algoritmul Fisher-Yates pentru a permuta elementele unei liste în mod aleatoriu. Această funcție transformă lista într-un vector pentru a permite accesul și schimbarea elementelor în timp liniar, după care rezultatul este convertit înapoi într-o listă pentru a menține consistența cu restul sistemului. Randomizarea este inițializată folosind starea globală `*random-state*`, asigurând unicitatea puzzle-urilor la fiecare rulare.

4.1.7 Strategia de generare a soluției complete

Procesul de generare nu încearcă să construiască un puzzle direct, ci pornește de la o soluție completă validă. Funcția `fill-random` orchestrează acest proces astfel:

- **Inițializarea:** se pornește de la o tablă goală creată prin `create-empty-board`.
- **Popularea parțială:** primul rând al tablei este completat cu o permutare aleatorie a valorilor posibile (1–4 sau 1–9).
- **Finalizarea prin Backtracking:** se apelează funcția `solve` pe această tablă parțial completată pentru a genera restul valorilor conform regulilor Sudoku.

Funcția `generate-solution` acționează ca un wrapper care garantează returnarea unei table complete, iterând procesul în cazul (rar) în care solver-ul nu poate găsi o soluție pentru configurația inițială.

4.1.8 Transformarea soluției în puzzle

Odată obținută o tablă completă, aceasta este transformată într-o provocare prin eliminarea unui număr specific de valori (*Hole Punching*).

- **Selecția pozițiilor:** funcția `generate-puzzle` generează o listă cu toate coordonatele posibile ale tablei, le amestecă aleatoriu și selectează primele `num-to-remove` poziții.
- **Abordarea funcțională:** eliminarea efectivă a valorilor este realizată de funcția `remove-cells`, care utilizează mecanismul `reduce`. Aceasta aplică succesiv funcția `set-value` pe tabla de joc, generând la fiecare pas o nouă instanță a tablei cu valoarea 0 (celulă goală) la coordonatele specificate, respectând astfel principiul imutabilității datelor.

4.1.9 Puzzle demo

Integrarea tuturor componentelor este vizibilă în funcția `run-demo` se apelează din fișierul `main.lisp`. Aceasta oferă un flux complet: generează un puzzle de 4x4 cu 8 spații goale și unul de 9x9 cu 40 de spații goale, afișând starea inițială a puzzle-ului și pașii de rezolvare parcurși de algoritmul de backtracking până la obținerea soluției finale.

4.2 Implementarea în Haskell

Implementarea în Haskell se sprijină pe un sistem de tipuri static, riguros și pe o separare clară între funcțiile pure și cele care produc efecte secundare. Soluția a fost divizată modular în patru componente principale: definirea regulilor (`Sudoku.hs`), algoritmul de rezolvare (`Solver.hs`), generatorul de puzzle-uri (`Generator.hs`) și evaluarea performanței (`Benchmark.hs`).

4.2.1 Reprezentarea datelor și validarea regulilor

În fișierul `Sudoku.hs`, tabla de joc este definită printr-un sinonim de tip, `type Board = [[Int]]`, păstrând imutabilitatea structurii. Pentru reprezentarea corectă a acesteia au fost implementate următoarele metode:

- **Validarea:** Funcțiile `validRow`, `validCol` și `validBox` nu folosesc bucle explicite, ci se bazează pe funcții de ordin superior și list comprehensions pentru a filtra celulele goale și verifica unicitatea într-o manieră pur declarativă.
- **Identificarea celulelor libere:** Funcția `findEmpty` generează o listă a tuturor coordonatelor care conțin valoarea 0 (*list comprehension*), funcția aplică `listToMaybe` pentru a extrage doar primul element. Astfel, rezultatul este încapsulat în tipul `Maybe (Int, Int)`. Spre deosebire de `nil`-ul din Lisp, tipul `Maybe` forțează compilatorul să verifice dacă programatorul tratează atât cazul în care s-a găsit o celulă (`Just`), cât și cazul în care tabla este plină (`Nothing`).

4.2.2 Backtracking recursiv: Maybe Monad și Pattern Matching

Logica de rezolvare din `Solver.hs` ilustrează nucleul programării funcționale în Haskell, eliminând complet instrucțiunile de control de tip `if-else` în favoarea *Pattern Matching*-ului și propagării monadice.

Funcția principală `solve :: Board -> Maybe Board` analizează rezultatul dat de funcția `findEmpty` direct prin `case ... of`:

- Dacă returnează `Nothing`, înseamnă că nu mai există celule goale, iar funcția împachează tabla curentă ca soluție finală: `Just board`.
- Dacă returnează `Just (r, c)`, delegă rezolvarea către funcția auxiliară `tryValues`, pasând lista tuturor valorilor posibile `[1 .. boardSize board]`.

Funcția `tryValues` utilizează *Pattern Matching* pe lista de valori `(v:vs)`. Pentru fiecare valoare `v`, se apelează `tryValue`. Aici, monada `Maybe` automatizează backtracking-ul: dacă `tryValue` returnează `Just solution`, succesul este propagat imediat în sus pe lanțul recursiv. Dacă returnează `Nothing`, funcția face *fallback* și se autoapelează recursiv pentru restul valorilor `(vs)`.

4.2.3 Generarea puzzle-urilor: randomizare și IO Monad

Deoarece Haskell este un limbaj pur, funcțiile nu pot genera efecte secundare fără a marca acest lucru explicit în semnătura tipului. Fișierul `Generator.hs` gestionează această complexitate utilizând `IO Monad`.

- **Randomizarea și izolarea efectelor:** Orice operațiune care implică hazard, precum funcția `shuffle` (care amestecă o listă folosind `randomRIO`), este marcată explicit în semnătura sa prin tipul `IO [a]`. Această abordare izolează "impuritatea" și protejează restul sistemului, permițând compilatorului să verifice riguros unde și cum interacționează starea sistemului cu logica jocului.
- **Strategia "Top-Down":** Generatorul nu construiește un puzzle pornind de la zero, ci aplică o strategie prin care creează întâi o soluție de bază. Funcția `fillRandom` rulează în contextul `IO` pentru a obține o permutare aleatorie a primului rând, dar folosește o funcție de ordin superior pură (`foldr`) pentru a insera aceste valori pe o tablă goală. Ulterior, delegarea către solver-ul principal (`solve`) completează instantaneu restul tablei, garantând obținerea unei soluții de referință 100% valide.
- **Metoda "Hole Punching":** Transformarea soluției într-un puzzle jucabil se realizează prin eliminarea unui număr predefinit de celule. Funcția `generatePuzzle` generează toate coordonatele posibile ale tablei, le amestecă (în `IO`) și selectează primele `N` elemente. Acestea sunt transmise apoi către funcția `removeCells`, care utilizează din nou `foldr` pentru a înlocui valorile respective cu 0 (celule goale).

4.2.4 Claritatea codului și apropierea de specificația matematică

Un avantaj distinct al implementării Haskell este că semnăturile de tip funcționează ca o documentație statică, verificată automat de compilator:

```
valid      :: Board -> Int -> Int -> Int -> Bool
findEmpty  :: Board -> Maybe (Int, Int)
setValue   :: Board -> Int -> Int -> Int -> Board
solve      :: Board -> Maybe Board
```

Un cititor înțelege contractul fiecărei funcții fără a parcurge implementarea. Mai mult, orice discrepanță între semnătură și corp produce o eroare la compilare, eliminând o categorie întreagă de erori posibile — avantaj absent în Common Lisp, unde tipurile sunt verificate exclusiv la rulare.

Al doilea element de claritate îl reprezintă list comprehension-urile, care exprimă operațiile pe colecții într-o manieră apropiată de notația matematică set-builder. Extragerea valorilor dintr-o sub-grilă în `validBox` se scrie astfel:

```
boxValues =
  [ board !! r !! c
  | r <- [boxRow .. boxRow + bSize - 1]
  , c <- [boxCol .. boxCol + bSize - 1]
  ]
```

Expresia se citește direct ca o definiție matematică: „mulțimea valorilor de pe pozițiile (r, c) pentru r și c parcurgând intervalele specificate”. Echivalentul imperativ ar necesita două bucle imbricate și o variabilă acumulator explicită. În mod similar, `validRow` se reduce la o singură linie:

```
validRow board row val =
  val 'notElem' filter (/= 0) (board !! row)
```

Linia comunică direct intenția: „valoarea nu se află printre elementele nenule ale rândului”, fără cod auxiliar.

Aceste caracteristici fac ca implementarea Haskell să se comporte ca o *specificație executabilă* a problemei: codul descrie **ce** trebuie calculat, nu **cum**, ceea ce este deosebit de potrivit pentru o problemă cu invariante matematice clare precum Sudoku.

4.2.5 Alte observații și detalii de implementare

Un aspect tehnic demn de menționat este modularitatea facilitată de Haskell, vizibilă în modulul de testare a performanței (`Benchmark.hs`). Utilizarea funcției `replicateM` din `Control.Monad` permite rularea repetată a solver-ului pentru calcularea unui timp mediu de execuție, demonstrând cum structurile de control din Haskell sunt ele însele funcții de bibliotecă.

5 Comparații între abordarea Common Lisp și Haskell

5.1 Modul de rezolvare prin Backtracking

Criteriu de comparație	Common Lisp (CL)	Haskell (HS)
Paradigma generală	Procedural-funcțională, bazată pe recursivitate explicită.	Declarativă, bazată pe abstracții matematice (Monade și Pattern Matching).
Controlul fluxului (Plumbing)	Manual. Programatorul trebuie să scrie instrucțiunile care dictează <i>cum</i> se trece de la un pas la altul.	Automatizat. Sistemul monadic (ex. List Monad) se ocupă în spate de iterarea prin posibilități.
Ramificarea posibilităților	Folosește structuri iterative explicite (ex: <code>loop for val from 1 to size</code>). Încearcă fiecare valoare secvențial.	Exprimă calcul non-determinist. Conceptul spune: „explorează simultan toate valorile din lista <code>[1..size]</code> ”.
Pruning (Tăierea ramurilor invalide)	Se face prin instrucțiuni condiționale clasice: (<code>when (valid? ...) ...</code>) oprește generarea dacă valoarea nu e bună.	Se face prin <code>guard</code> sau filtrare în <i>list comprehensions</i> . Dacă nu e valid, ramura devine lista goală <code>[]</code> .
Propagarea eșecului (Backtracking)	Explicită. Când se ajunge într-un punct mort, funcția returnează manual <code>nil</code> . Funcția-părinte prinde acest <code>nil</code> și continuă bucla.	Implicită. Ramurile invalide (<code>[]</code> în List Monad sau <code>Nothing</code> în Maybe) pur și simplu "mor" silențios. Nu necesită cod manual de întoarcere.
Returnarea soluției finale	Soluția găsită urcă manual înapoi pe lanțul de recursivitate	Soluția validă este pur și simplu "filtrată" și împachetată automat în contextul monadic la finalul execuției.

Tabela 1: Comparație abordare Backtracking: Common Lisp vs. Haskell

5.2 Comparație: tratarea succesului/eșecului

Aspect	Common Lisp (NIL)	Haskell (Maybe)
Paradigmă	Convenție dinamică (fals = nil)	Tip algebric de date strict
Reprezentare Eșec	<code>nil</code> (simbol universal)	<code>Nothing</code> (constructor de tip)
Reprezentare Succes	Obiectul returnat direct (Board)	<code>Just Board</code> (valoare împachetată)
Verificare	Condițională simplă: (<code>if result ...</code>).	Pattern matching obligatoriu
Siguranță	Permisivitate; risc de confuzie cu lista goală.	Siguranță ridicată; erorile sunt detectate la compilare

Tabela 2: Comparație: Tratarea Succesului/Eșecului (CL vs. HS)

6 Testarea sistemului

6.1 Suita de teste

Testarea sistemului a fost realizată independent pentru fiecare implementare, utilizând abordări specifice fiecărui limbaj. Ambele suite acoperă aceleași categorii logice: validarea regulilor de bază, identificarea celulelor libere, comportamentul solver-ului și performanța de execuție.

6.1.1 Teste Common Lisp

Fișierul `tests.lisp` este organizat în trei funcții de test distincte, apelate secvențial la rulare.

Teste unitare (`run-all-tests`): sunt verificate zece scenarii individuale prin compararea rezultatului funcției `solve` cu valoarea așteptată:

- **Test 1** – Tablă deja completă (4×4): `solve` trebuie să returneze aceeași tablă neschimbată.
- **Test 2** – Tablă goală (4×4): `solve` trebuie să returneze o soluție validă (`non-nil`).
- **Test 3** – Tablă parțial completată validă (4×4): `solve` trebuie să găsească o soluție.
- **Test 4** – Tablă invalidă (4×4 , conține duplicat pe coloană): `solve` trebuie să returneze `nil`.
- **Test 5** – Tablă parțial completată validă (9×9 , puzzle clasic): `solve` trebuie să găsească o soluție.
- **Test 6** – Tablă invalidă (9×9 , conține duplicat pe primul rând): `solve` trebuie să returneze `nil`.
- **Testele 7–10** – Validarea funcției `valid?` pe o tablă 4×4 aproape completă: se verifică că o valoare absentă pe rând este permisă (Test 7), că un duplicat pe rând este respins (Test 8), că un duplicat pe coloană este respins (Test 9) și că un duplicat în sub-grilă este respins (Test 10).

Teste pentru generator (`run-generator-tests`): sunt verificate trei proprietăți ale funcției `generate-4x4`:

- **Test 1** – Puzzle-ul generat cu 6 celule goale este rezolvabil (apelând `solve` pe el).
- **Test 2** – Soluția obținută prin `solve` este identică cu soluția de referință returnată de generator.
- **Test 3** – Puzzle-ul generat cu 8 celule goale conține exact 8 valori egale cu 0.

Teste de performanță (`run-performance-tests`): se măsoară timpul de execuție al funcției `solve` utilizând macro-ul `time` din SBCL, pentru o tablă 4×4 complet goală și pentru tabla standard 9×9 .

6.1.2 Teste Haskell

Fișierul `Tests.hs` utilizează framework-urile `HUnit` (teste unitare deterministe) și `QuickCheck` (teste bazate pe proprietăți), organizate în cinci grupuri.

Grupul `valid` (4 teste `HUnit`): testează funcția de validare a plasării unei cifre:

- Valoare absentă din rând, coloană și sub-grilă — permisă (`valid complete4x4 0 0 5`).
- Valoare duplicat pe rând — respinsă (`valid complete4x4 0 0 1`).
- Valoare duplicat pe coloană — respinsă (`valid complete4x4 0 0 3`).
- Valoare duplicat în sub-grilă 2×2 — respinsă (`valid complete4x4 0 0 2`).

Grupul `findEmpty` (3 teste `HUnit`): testează identificarea primei celule libere:

- Tablă parțial completată: prima celulă goală este `Just (0,1)`.
- Tablă completă: rezultat `Nothing`.
- Tablă complet goală: prima celulă goală este `Just (0,0)`.

Grupul `setValue` (3 teste `HUnit`): testează modificarea imutabilă a tablei:

- Valoarea este plasată corect la coordonatele specificate.
- Celelalte celule rămân nemodificate.
- Dimensiunile tablei sunt păstrate după modificare.

Grupul `boardSize` (2 teste `HUnit`): verifică că funcția `boardSize` returnează 4 pentru o tablă 4×4 și 9 pentru o tablă 9×9 .

Grupul `solve` (8 teste `HUnit`): testează solver-ul pe scenarii variate:

- Puzzle 4×4 valid: `solve` returnează `Just solution`.
- Soluție exactă: rezultatul coincide cu soluția unică predefinită `solution4x4`.
- Puzzle imposibil (două valori identice pe același rând): `solve` returnează `Nothing`.
- Tablă goală 4×4 : `solve` găsește o soluție.
- Tablă completă: `solve` o returnează nemodificată.
- Puzzle 9×9 clasic: `solve` găsește o soluție.
- Soluția nu conține celule goale (niciun 0 în lista aplatizată).
- Soluția este validă — niciun duplicat pe rânduri sau coloane.

Teste `QuickCheck` (2 proprietăți): sunt verificate proprietăți invariante ale solver-ului, independent de un caz concret:

- `prop_solveNoZeros`: soluția returnată pentru `puzzle4x4` nu conține valori 0.
- `prop_solveSameSize`: tabla soluție păstrează dimensiunile 4×4 .

6.2 Rezultate și execuție

6.2.1 Rezultate Common Lisp (SBCL 2.6.3)

Toate testele unitare și testele pentru generator au trecut cu succes:

```
--- UNIT TESTS ---
Test 1 - Full table: PASSED
Test 2 - Empty table for 4x4: PASSED
Test 3 - Valid board for 4x4: PASSED
Test 4 - Invalid board for 4x4: PASSED
Test 5 - Valid board for 9x9: PASSED
Test 6 - Invalid board for 9x9: PASSED
Test 7 - Valid value on row? PASSED
Test 8 - Duplicate on row? PASSED
Test 9 - Duplicate on column? PASSED
Test 10 - Duplicate in box? PASSED

--- GENERATOR TESTS ---
Test 1 - Generated 4x4 is solvable: PASSED
Test 2 - Solution matches original: PASSED
Test 3 - Correct number of empty cells (8): PASSED

--- PERFORMANCE TESTS - EXECUTION TIME ---
Performance for sudoku 4x4 (empty table):
Evaluation took: 0.000 seconds of real time

Performance for sudoku 9x9 (standard table):
Evaluation took: 0.001 seconds of real time
```

Solver-ul Common Lisp rezolvă tabla 4×4 într-un timp neglijabil, iar tabla 9×9 în sub-milisecundă, datorită pruning-ului agresiv realizat de algoritmul de backtracking combinat cu validarea imediată prin `valid?`.

6.2.2 Rezultate Haskell (GHC 9.6.7)

Toate cele 20 de teste HUnit au trecut, iar proprietățile QuickCheck au fost verificate pe 100 de cazuri generate automat:

```
=== HUnit Tests ===

Cases: 20  Tried: 20  Errors: 0  Failures: 0
=== QuickCheck Tests ===
prop_solveNoZeros:
+++ OK, passed 100 tests.
prop_solveSameSize:
+++ OK, passed 100 tests.
```

6.3 Evaluare comparativă și concluzii

Cele două suite de teste reflectă filozofiile diferite ale celor două limbaje și oferă o perspectivă complementară asupra calității implementărilor.

Din punct de vedere al **expresivității testelor**, suita Haskell este mai structurată și mai exhaustivă: grupurile HUnit separă clar responsabilitățile fiecărei funcții (`valid`, `findEmpty`, `setValue`, `boardSize`, `solve`), iar QuickCheck adaugă un nivel suplimentar de verificare prin generarea automată de cazuri de test. Suita Common Lisp este mai directă și mai procedurală — testele sunt scrise explicit, fără framework extern, ceea ce face codul mai ușor de urmărit, dar mai puțin extensibil.

Din punct de vedere al **siguranței**, Haskell oferă un avantaj structural: sistemul de tipuri garantează la compilare că funcțiile `solve` și `findEmpty` tratează obligatoriu ambele cazuri (`Just/Nothing`), eliminând o clasă întreagă de erori înainte ca testele să ruleze. În Common Lisp, aceeași garanție este asigurată exclusiv prin disciplina programatorului și prin testele explicite (Testele 4, 6).

Din punct de vedere al **performanței**, ambele implementări se comportă excelent pentru dimensiunile testate. Tabla 9×9 este rezolvată sub o milisecundă în ambele cazuri, ceea ce confirmă eficiența algoritmului de backtracking cu pruning pe puzzle-uri standard. Haskell beneficiază suplimentar de evaluarea leneșă, care evită generarea ramurilor invalide din arborele de căutare înainte ca acestea să fie necesare.

În concluzie, ambele implementări sunt corecte și robuste. Common Lisp favorizează un stil de testare pragmatic și interactiv, potrivit pentru iterații rapide în REPL. Haskell favorizează un stil declarativ și structurat, în care corectitudinea este parțial garantată de compilator, iar testele validează comportamentul la un nivel mai înalt de abstractizare.

Bibliografie

- [1] Seibel, P. (2005). *Practical Common Lisp*. Apress. Disponibil online: <http://www.gigamonkeys.com/book>
- [2] SBCL Team. (2024). *SBCL User Manual, Version 2.6.3*. Steel Bank Common Lisp. <http://www.sbcl.org/manual/>
- [3] Abelson, H., Sussman, G. J., & Sussman, J. (1996). *Structure and Interpretation of Computer Programs* (2nd ed.). MIT Press.
- [4] Lipovača, M. (2011). *Learn You a Haskell for Great Good!* No Starch Press. Disponibil online: <http://learnyouahaskell.com>
- [5] GHC Team. (2024). *GHC User's Guide, Version 9.6.7*. https://downloads.haskell.org/ghc/latest/docs/users_guide/
- [6] Claessen, K., & Hughes, J. (2000). QuickCheck: A lightweight tool for random testing of Haskell programs. *ACM SIGPLAN Notices*, 35(9), 268–279.